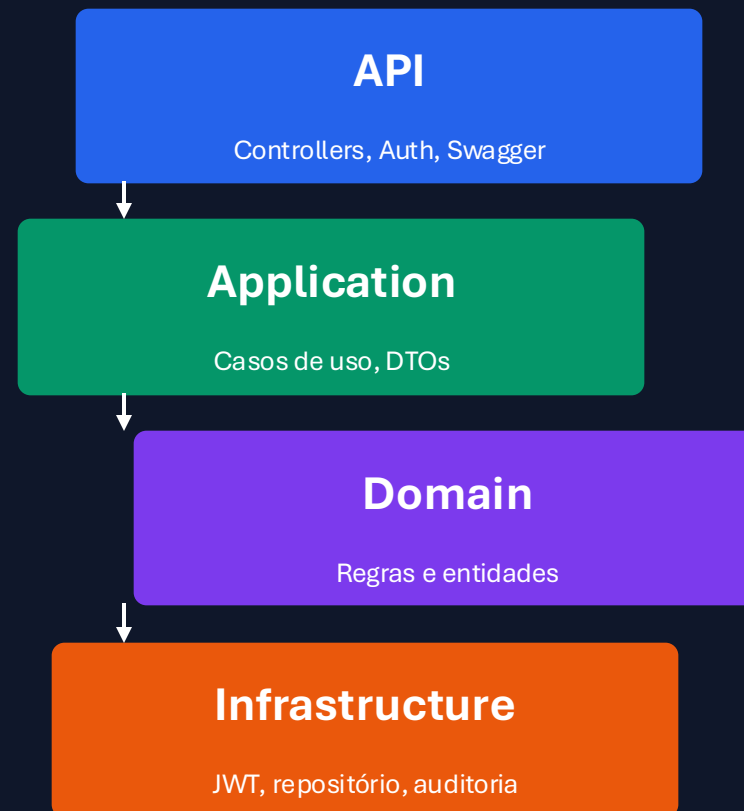


.NET Core

Backend seguro e pronto para produção

JWT • Roles • Claims • Policies • CORS • Rate Limit • Middlewares • Métricas • Health • Swagger • 4 camadas

Prof. Lucas Lima



Objetivo da aula de 5 horas

Construir uma visão completa de backend moderno usando um sistema simples de estoque.

1

Arquitetura em 4 camadas

Separar API, Application, Domain e Infrastructure para organizar responsabilidades.

2

Segurança com JWT

Entender login, token, roles, claims, policies, HTTPS, CORS e segredos.

3

Qualidade da API

Swagger/OpenAPI, versionamento, contratos, filtros, ordenação e erros padronizados.

4

Resiliência e proteção

Rate limiting, middlewares, tratamento global de erro e auditoria.

5

Observabilidade

Health check, métricas e prontidão para produção.

Sistema exemplo: Estoque

Domínio simples para explicar conceitos complexos sem distrair os alunos.



Consulta

Usuários autenticados podem consultar produtos, filtrar por nome e ordenar por preço/quantidade.



Edição

Somente Estoquista/Admin com a claim certa pode editar quantidade ou preço.



Auditoria

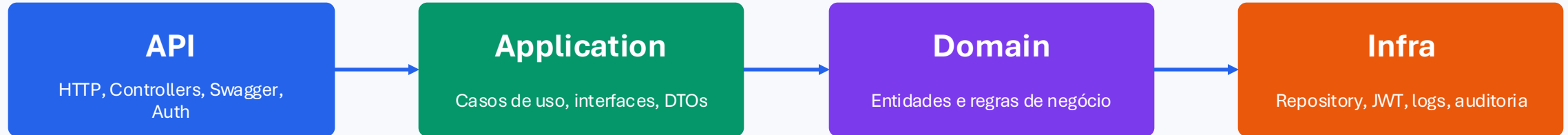
Cada alteração relevante deve registrar quem fez, quando fez e em qual recurso.

```
GET  /api/v1/produtos?nome=mouse&ordenarPor=preco
PUT  /api/v1/produtos/{id}
POST /api/auth/login
GET  /health
GET  /metrics
```

Mensagem principal: usamos um domínio pequeno para mostrar práticas reais de backend.

Visão macro da arquitetura

Cada camada tem uma responsabilidade clara.



Regra de ouro

Domain não depende de ninguém. Ele representa o coração do negócio.



Application

Orquestra o caso de uso, mas não sabe se existe SQL, MongoDB ou API externa.



Infrastructure

Implementa detalhes técnicos que podem ser trocados no futuro.

Por que separar em camadas?

Separação reduz acoplamento e melhora teste, manutenção e evolução.

API

Recebe HTTP, valida entrada superficial, chama Application e retorna resposta.

Application

Implementa casos de uso: consultar, editar, validar fluxo e regras de aplicação.

Domain

Entidades, regras fundamentais e linguagem do negócio.

Infra

Banco, JWT, e-mail, cache, logs, provedores externos.

Controller NÃO deve saber SQL.
Application NÃO deve saber detalhes de JWT.
Domain NÃO deve saber que existe Swagger, HTTP ou banco.

Essa separação deixa a aula mais limpa e aproxima o projeto de uma arquitetura real.

Domain: o coração do negócio

Aqui ficam conceitos que existiriam mesmo sem API, banco ou framework.

CAMADA 1

```
namespace Estoque.Domain.Entidades;

public class Produto
{
    public Guid Id { get; set; }
    public string Nome { get; set; } =
string.Empty;
    public decimal Preco { get; set; }
    public int Quantidade { get; set; }
}
```



Como explicar

Produto é conceito do negócio. Ele não nasce porque existe controller ou banco.



Exemplos no Domain

Produto, Usuario, Papel, ClaimsEstoque, regras simples e validações do domínio.



Evite no Domain

Entity Framework, Swagger, HttpContext, JWT, conexão de banco.

Application: casos de uso

CAMADA 2

A camada de Application responde: o que o sistema sabe fazer?

Interfaces

Define contratos como IProdutoRepository e ITokenService.

DTOs

Define entrada e saída da aplicação sem expor diretamente o domínio.

Services

Executa casos de uso: consultar estoque, editar produto, autenticar usuário.

```
public interface ITokenService
{
    (string Token, DateTime ExpiraEmUtc) Gerar(Usuario usuario);
}
```

Application conhece abstrações. Infra implementa os detalhes.

Infrastructure: detalhes técnicos

CAMADA 3

Implementa o que a Application pediu através de interfaces.



Repository

Pode ser in-memory na aula, SQL Server em produção, MongoDB em outro cenário.



JwtTokenService

Gera token, assina com chave e devolve expiração.



Auditoria

Registra ações importantes como edição de estoque.

```
services.AddScoped<IProdutoRepository, ProdutoRepository>();  
services.AddScoped<ITokenService, JwtTokenService>();  
services.AddScoped<IAuditoriaService, AuditoriaService>();
```

Em produção, é aqui que moram integrações com recursos externos.

API: entrada e saída HTTP

CAMADA 4

A API conversa com clientes externos: front-end, mobile, gateway e outros sistemas.



Controllers

Recebem request, chamam Application e retornam status HTTP.



Autorização

Usa [Authorize], roles, claims e policies.



Documentação

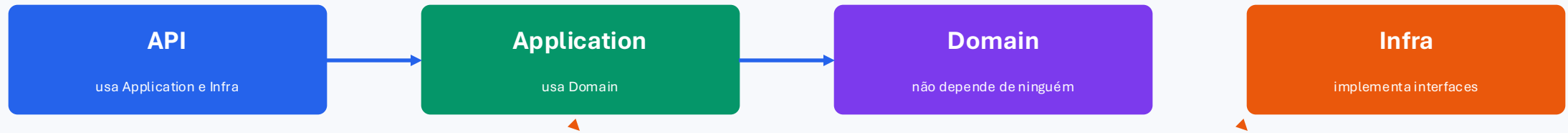
Swagger/OpenAPI permite explorar e testar a API.

```
[HttpPut("{id:guid}")]  
[Authorize(Policy = Politicas.PodeEditar)]  
public async Task<IActionResult> Editar(Guid id, EditarProdutoRequest request)
```

Controller deve ser fino: pouca regra, muita delegação para Application.

Dependência correta entre camadas

As setas indicam quem pode conhecer quem.



```
API -> Application
API -> Infrastructure
Infrastructure -> Application
Infrastructure -> Domain
Application -> Domain
Domain -> ninguém
```

Comandos iniciais

Criando solution, projetos e referências.

```
mkdir stocksystem
cd stocksystem

dotnet new sln -n StockSystem
mkdir src

dotnet new classlib -n Estoque.Domain -o src/Estoque.Domain -f net9.0
dotnet new classlib -n Estoque.Application -o src/Estoque.Application -f net9.0
dotnet new classlib -n Estoque.Infra -o src/Estoque.Infra -f net9.0
dotnet new webapi -n Estoque.Api -o src/Estoque.Api -f net9.0 --use-controllers
```

```
dotnet sln add src/Estoque.Domain/Estoque.Domain.csproj
dotnet sln add src/Estoque.Application/Estoque.Application.csproj
dotnet sln add src/Estoque.Infra/Estoque.Infra.csproj
dotnet sln add src/Estoque.Api/Estoque.Api.csproj

dotnet add src/Estoque.Application reference src/Estoque.Domain
dotnet add src/Estoque.Infra reference src/Estoque.Application
dotnet add src/Estoque.Infra reference src/Estoque.Domain
dotnet add src/Estoque.Api reference src/Estoque.Application
dotnet add src/Estoque.Api reference src/Estoque.Infra
```

O que é o Program.cs?

Ele é o ponto de montagem da aplicação ASP.NET Core.



builder.Services

Registra dependências: controllers, JWT, CORS, Swagger, health, middlewares e serviços.



builder.Build()

Finaliza a configuração e cria a aplicação que será executada.



app.Use / app.Map

Define o pipeline HTTP e quais endpoints ficam disponíveis.

```
builder.Services.AddControllers();  
var app = builder.Build();  
app.UseAuthentication();  
app.MapControllers();  
app.Run();
```

Pipeline HTTP: a ordem importa

Cada request passa por etapas antes de chegar ao Controller.



Authentication

Lê o Bearer token, valida o JWT e preenche HttpContext.User.



Authorization

Aplica [Authorize], roles, claims e policies.

Regra prática: primeiro autentica, depois autoriza.

HTTPS e HSTS

Segurança de transporte: proteger a comunicação entre cliente e API.



HTTP

Dados trafegam sem proteção de transporte. Token e dados sensíveis ficam expostos na rede.



HTTPS

Usa TLS para criptografar a comunicação entre cliente e servidor.

```
app.UseHttpsRedirection();  
app.UseHsts();
```



HSTS

Informa ao navegador que aquele domínio deve ser acessado somente por HTTPS por um período.

JWT não substitui HTTPS. JWT identifica o usuário; HTTPS protege o caminho.

Configuração e gestão de segredos

Configuração pode ir no appsettings; segredo não deve ir para o Git.



Configuração

LogLevel, URLs, feature flags, timeouts e opções não sensíveis.



Segredo

JWT Key, connection string, senha, client secret e API key.



Produção

Use Key Vault, Secrets Manager, Kubernetes Secrets e variáveis de ambiente.

```
dotnet user-secrets init  
dotnet user-secrets set "Jwt:Key" "sua-chave-longa-com-mais-de-32-  
caracteres"
```

Frase para a turma: appsettings é configuração; cofre é para segredo.

Vulnerabilidades comuns e proteções

Uma API pronta para produção precisa olhar além do “funciona na minha máquina”.

Segredo no Git

User Secrets / Key Vault

Endpoint aberto

[Authorize] e policies

Abuso / brute force

Rate limiting

Erro expõe stack trace

Middleware + ProblemDetails



Sem rastreabilidade

Auditoria

Nunca confie apenas no front-end.
Valide e autorize no back-end.

Qual problema o JWT resolve?

Como a API sabe quem está chamando cada endpoint?



Sem token

A API recebe uma request, mas não sabe quem é o usuário.



Login

Usuário envia credenciais para um serviço de autenticação.



Com JWT

Cada request traz um cartão assinado no header Authorization.

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6...
```

JWT é stateless: o servidor não precisa guardar sessão para cada usuário.

Fluxo do JWT na prática

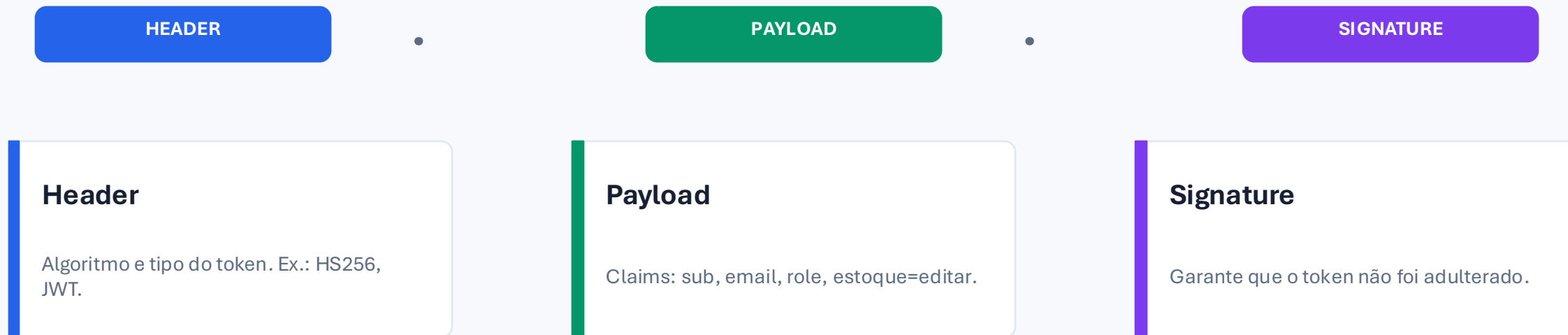
Login gera token; chamadas seguintes usam o token.



1. POST /login → retorna accessToken
2. GET /api/v1/produtos com Authorization: Bearer <token>
3. API valida assinatura, expiração, issuer e audience
4. API lê role/claims e aplica policies

Anatomia de um JWT

Um JWT possui três partes separadas por ponto.



Atenção: JWT assinado não é o mesmo que JWT criptografado. Não coloque senha ou segredo dentro dele.

JwtTokenService: onde o token nasce

Recebe um Usuario, monta claims, define expiração e assina.

```
public (string Token, DateTime ExpiraEmUtc) Gerar(Usuario usuario)
{
    var issuer = _config["Jwt:Issuer"]!;
    var audience = _config["Jwt:Audience"]!;
    var key = _config["Jwt:Key"]!;
    var expira = DateTime.UtcNow.AddMinutes(60);

    var claims = new List<Claim>
    {
        new(JwtRegisteredClaimNames.Sub, usuario.Id.ToString()),
        new(JwtRegisteredClaimNames.Email, usuario.Email),
        new(ClaimTypes.Role, usuario.Papel)
    };
}
```



Responsabilidade

Gerar o token, não validar senha diretamente.



Produção

Autenticação pode vir de Identity, Entra ID, Keycloak ou outro provedor.



Segurança

A chave de assinatura deve vir de um lugar seguro.

Validação JWT no Program.cs

A API precisa validar o token que recebe.

```
builder.Services
    .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters =
            new TokenValidationParameters
            {
                ValidateIssuer = true,
                ValidateAudience = true,
                ValidateLifetime = true,
                ValidateIssuerSigningKey = true,
                ValidIssuer = builder.Configuration["Jwt:Issuer"],
                ValidAudience = builder.Configuration["Jwt:Audience"],
                IssuerSigningKey = new SymmetricSecurityKey(
                    Encoding.UTF8.GetBytes(jwtKey))
            };
    });
```

Issuer

Quem criou o token?

Audience

Para quem o token foi criado?

Lifetime

O token ainda não expirou?

Signing Key

A assinatura está válida?

Authentication vs Authorization

Duas perguntas diferentes.



Authentication

Quem é você?

Exemplo: token válido pertence ao usuário admin@fiap.com.



Authorization

O que você pode fazer?

Exemplo: usuário pode editar estoque?

```
app.UseAuthentication(); // lê o Bearer token e monta HttpContext.User  
app.UseAuthorization(); // aplica [Authorize], roles, claims e policies
```

Role, Claim e Policy

Os três trabalham juntos, mas não significam a mesma coisa.



Role

Grupo ou papel do usuário.
Ex.: Consulta, Estoquista, Admin.



Claim

Informação ou permissão carregada no token.
Ex.: estoque=editar.



Policy

Regra que combina autenticação, role e claim.

```
PodeEditarEstoque = usuário autenticado  
                    AND (Role Estoquista OR Role Admin)  
                    AND Claim estoque = editar
```

Exemplo de Roles no estoque

Role responde: qual é o papel desse usuário no sistema?



Consulta

Pode consultar estoque, mas não altera dados.



Estoquista

Pode consultar e editar estoque quando possuir permissão.



Admin

Pode acessar áreas administrativas, mas ainda pode depender de claims.

```
new Claim(ClaimTypes.Role, Papeis.Admin)

[Authorize(Roles = "Admin")]
```

Role ajuda a agrupar usuários. Não use Role para representar todas as permissões pequenas.

Claims de negócio

Claims respondem: quais informações ou permissões esse usuário possui?

JWT Payload

```
{  
  "sub": "10",  
  "email": "admin@fiap.com",  
  "role": "Admin",  
  "estoque": "editar",  
  "filial": "SP"  
}
```



Claim técnica

sub, email, name, role.



Claim de negócio

estoque=editar, filial=SP, departamento=TI.



Uso

Policies verificam essas claims para liberar ou bloquear endpoints.

Policies: regras de autorização

Policy centraliza a regra e evita if de permissão dentro do controller.

```
public static class Politicas
{
    public const string PodeConsultar = "PodeConsultarEstoque";
    public const string PodeEditar = "PodeEditarEstoque";

    public static void Configurar(AuthorizationOptions options)
    {
        options.AddPolicy(PodeConsultar, policy =>
            policy.RequireAuthenticatedUser()
                .RequireRole(Papeis.Consulta, Papeis.Estoquista, Papeis.Admin));

        options.AddPolicy(PodeEditar, policy =>
            policy.RequireAuthenticatedUser()
                .RequireRole(Papeis.Estoquista, Papeis.Admin)
                .RequireClaim("estoque", "editar"));
    }
}
```

PodeConsultar

Autenticado AND Role Consulta/Estoquista/Admin.

PodeEditar

Autenticado AND Role Estoquista/Admin AND claim estoque=editar.

No controller

[Authorize(Policy = Politicas.PodeEditar)]

401 vs 403

Essa diferença ajuda os alunos a entender autenticação e autorização.



401 Unauthorized

A API não sabe quem você é.

Causas comuns:

- sem token
- token inválido
- token expirado



403 Forbidden

A API sabe quem você é, mas você não tem permissão.

Exemplo:

Usuário Consulta tenta editar estoque.

Frase para a aula: 401 é identidade ausente/inválida; 403 é identidade válida sem permissão.

Controller protegido

O Controller usa as policies sem repetir regra de segurança.

```
[ApiController]
[ApiVersion("1.0")]
[Route("api/v{version:apiVersion}/produtos")]
[Authorize]
public class ProdutosController : ControllerBase
{
    [HttpGet]
    [Authorize(Policy = Politicas.PodeConsultar)]
    public IActionResult Consultar() => Ok();

    [HttpPut("{id:guid}")]
    [Authorize(Policy = Politicas.PodeEditar)]
    public IActionResult Editar(Guid id) => NoContent();
}
```

[Authorize]

Exige usuário autenticado.

Policy

Exige regra composta.

Controller fino

Recebe, delega e responde.

Swagger / OpenAPI

Documentação que ajuda desenvolvimento, teste e integração.



OpenAPI

Especificação/contrato da API: endpoints, parâmetros, respostas e segurança.



Swagger UI

Interface visual para navegar e testar os endpoints.



JWT no Swagger

Permite informar Bearer token e testar endpoints protegidos.

```
builder.Services.AddSwaggerGen();
```

```
app.UseSwagger();  
app.UseSwaggerUI();
```

Não é só “tela bonita”: OpenAPI é um contrato entre backend e consumidores.

Versionamento de API

Permite evoluir sem quebrar clientes antigos.



URL versioning

/api/v1/produtos
/api/v2/produtos



Header versioning

X-API-Version: 1.0
Útil com gateways e clientes corporativos.

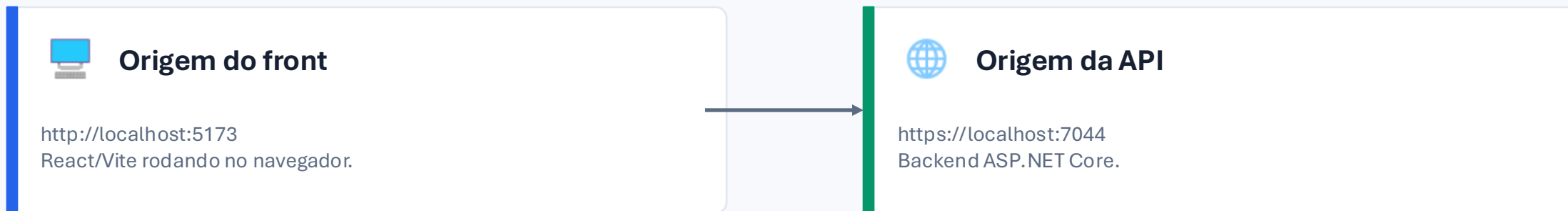
```
options.ApiVersionReader = ApiVersionReader.Combine(  
    new UriSegmentApiVersionReader(),  
    new HeaderApiVersionReader("X-API-Version"));
```

Mensagem: API pública ou consumida por muitos clientes precisa de estratégia de evolução.

O que é CORS?

CORS

Regra do navegador para controlar chamadas entre origens diferentes.



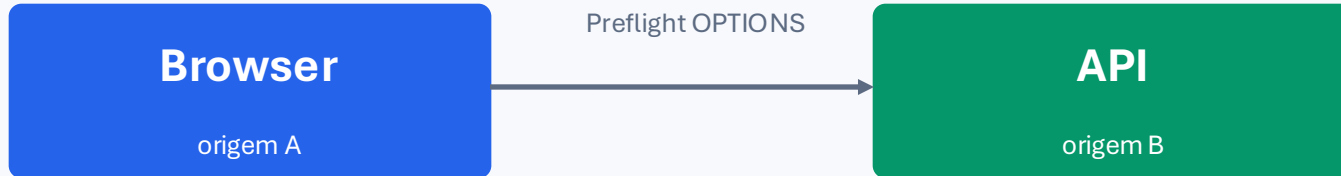
Origem = protocolo + domínio + porta

`http://localhost:5173` $\neq$ `https://localhost:7044`

CORS não autentica usuário. CORS controla chamadas feitas pelo browser.

CORS: o navegador protege o usuário

O bloqueio acontece no browser, não no Postman.



Quando aparece

Quando front e API estão em origens diferentes.

Quem decide

O navegador consulta a resposta da API.

Não confundir

Não substitui JWT, API Key ou autorização.

CORS no ASP.NET Core

Permitir apenas as origens necessárias.

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("FrontendAula", policy =>
    {
        policy.WithOrigins(
            "http://localhost:5173",
            "https://localhost:5173")
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});

app.UseCors("FrontendAula");
```

WithOrigins

Whitelist das origens permitidas.

AllowAnyHeader

Permite Authorization, Content-Type, X-API-Version etc.

Cuidado

Evite liberar tudo em produção sem necessidade.

Rate Limiting

PROTEÇÃO

Controla quantas requisições um cliente pode fazer em determinado tempo.



Problema

Brute force em login, scraping, abuso e picos artificiais.



Solução

Limitar tentativas por janela de tempo.



Resposta

Ao exceder o limite, retornar HTTP 429.

5 tentativas de login por minuto por IP
30 requisições de API por minuto por IP

Rate Limit no código

Podemos criar políticas diferentes para login e API.

```
builder.Services.AddRateLimiter(options =>
{
    options.RejectionStatusCode =
        StatusCodes.Status429TooManyRequests;

    options.AddFixedWindowLimiter("login", limiter =>
    {
        limiter.PermitLimit = 5;
        limiter.Window = TimeSpan.FromMinutes(1);
        limiter.QueueLimit = 0;
    });
});

app.UseRateLimiter();
```

Fixed Window

Conta requisições dentro de uma janela fixa de tempo.

Login

Limite mais restritivo para evitar brute-force.

API geral

Limite maior para proteger uso abusivo.

Middlewares

PIPELINE

Middleware é uma etapa no caminho da request e da response.



```
app.UseMiddleware<TratamentoErroMiddleware>();  
app.UseMiddleware<AuditoriaHttpMiddleware>();  
app.UseMiddleware<MetricasMiddleware>();
```

Middleware pode executar antes e depois do próximo componente do pipeline.

Ordem recomendada do pipeline

A ordem muda comportamento, segurança e logs.

```
app.UseHttpsRedirection();

app.UseMiddleware<TratamentoErroMiddleware>();
app.UseMiddleware<AuditoriaHttpMiddleware>();
app.UseMiddleware<MetricasMiddleware>();

app.UseCors("FrontendAula");
app.UseRateLimiter();

app.UseAuthentication();
app.UseAuthorization();

app.MapControllers();
```

Erro antes

Captura exceções dos próximos componentes.

Métricas e auditoria

Medem e registram o que aconteceu.

AuthN antes de AuthZ

Primeiro descobre quem é; depois decide se pode.

Tratamento de erros e ProblemDetails

Evite respostas inconsistentes e stack trace exposto.



Ruim

```
{ "message": "deu erro" }
```

ou stack trace retornado para o cliente.



Melhor

Resposta padronizada com title, status, detail e traceId.

```
{  
  "title": "Produto não encontrado",  
  "status": 404,  
  "detail": "Produto 123 não foi encontrado",  
  "traceId": "00-abcd..."  
}
```

Auditoria

Auditoria responde: quem fez o quê, quando e em qual recurso?



Log técnico

Timeout, exception, status code, tempo de resposta.



Auditoria

Usuário admin editou produto X às 20:15.



Produção

Enviar para App Insights, ELK, Splunk, Sentinel ou banco de auditoria.

```
AUDIT - User: admin@fiap.com | Operation: EDITAR_PRODUTO | Resource: produto-123
```

Métricas

OBSERVABILIDADE

Métricas mostram comportamento agregado da API.

12
34

Contagem

Quantas requests recebemos?



Latência

Quanto tempo a API demora?



Erros

Quantas falhas por endpoint/status?

```
GET /metrics
```

```
{ "requests": 120, "errors": 3, "produtosConsultados": 80 }
```

Logs contam histórias individuais. Métricas mostram padrões.

Health Check

Health responde se a aplicação está saudável agora.



Endpoint

GET /health



Consumidores

Kubernetes, Load Balancer, App Service, monitoramento.



Resultado

Healthy / Unhealthy e status HTTP.

```
builder.Services.AddHealthChecks().AddCheck<EstoqueHealthCheck>("estoque");  
app.MapHealthChecks("/health");
```

Filtros e ordenação

Recursos simples que melhoram muito a experiência de consumo da API.



Filtro

GET /api/v1/produtos?nome=mouse

Retorna somente produtos que combinam com o nome.



Ordenação

GET /api/v1/produtos?ordenarPor=preco

Retorna em ordem por preço.

```
if (!string.IsNullOrEmpty(nome))  
    produtos = produtos.Where(p => p.Nome.Contains(nome));  
  
produtos = ordenarPor == "preco" ? produtos.OrderBy(p => p.Preco) : produtos;
```

Contratos e DTOs

Controller não precisa receber ou devolver a entidade inteira do domínio.



Request DTO

Define exatamente o que o cliente pode enviar.



Response DTO

Define exatamente o que a API devolve.



Benefício

Evita exposição acidental de campos internos.

```
public record EditarProdutoRequest(decimal Preco, int
Quantidade);
```

Checklist de prontidão do backend

Funcionar não é o mesmo que estar pronto para produção.

- ✓ Arquitetura em camadas
- ✓ JWT configurado
- ✓ Roles/claims/policies
- ✓ CORS restrito
- ✓ HTTPS/HSTS
- ✓ Segredos fora do Git
- ✓ Swagger/OpenAPI
- ✓ Versionamento
- ✓ ProblemDetails
- ✓ Rate limiting
- ✓ Auditoria
- ✓ Health check
- ✓ Métricas
- ✓ Filtros/ordenação

Roteiro prático da aula

Uma sequência didática

1

Criar solution e projetos

Mostrar a estrutura antes de escrever muita regra.

2

Implementar Produto e casos de uso

Domain, Application e Infrastructure funcionando.

3

Subir API com Swagger

Testar endpoint aberto primeiro.

4

Adicionar JWT e Policies

Mostrar 401, 403 e token com claims.

5

Adicionar produção-ready

CORS, rate limit, middlewares, erros, health e métricas.

Demo 1: build e run

Mostrar o básico funcionando antes da segurança.

```
dotnet restore  
dotnet build  
dotnet run --project src/Estoque.Api/Estoque.Api.csproj
```



Resultado esperado

API sobe em <https://localhost:xxxx> e Swagger fica disponível.



Mensagem para os alunos

Primeiro fazemos funcionar; depois protegemos e observamos.

Demo 2: login e token

Gerar token com usuário Consulta, Estoquista ou Admin.

```
POST /api/auth/login
```

```
{  
  "email": "admin@fiap.com",  
  "senha": "123"  
}
```

Resposta:

```
{  
  "accessToken": "eyJhbGciOi...",  
  "expiresAt": "2026-...Z"  
}
```



Swagger Authorize

Colar o Bearer token para testar endpoints protegidos.



Decodificar token

Mostrar claims: sub, email, role, estoque=editar.

Demo 3: testar permissões

Fazer os alunos verem 200, 401, 403 e 204 na prática.

1 Sem token

GET produtos
→ 401

2 Consulta

GET produtos
→ 200

3 Consulta edita

PUT produto
→ 403

4 Admin edita

PUT produto
→ 204

```
Authorization: Bearer <token>  
PUT /api/v1/produtos/{id}
```

Essa demo consolida JWT, claims, role, policy e status HTTP.

O que muda em produção?

O exemplo é didático; a mentalidade precisa ser profissional.



Autenticação

Não usar usuário e senha hardcoded. Use provedor de identidade.



Segredos

JWT Key, connection string e secrets em cofre seguro.



Observabilidade

Logs, métricas, tracing e alertas centralizados.



Banco e transação

Trocar in-memory por banco real e validar concorrência.



Pipeline seguro

HTTPS, CORS restrito, rate limit, auditoria e tratamento de erros.

Exercícios

Pequenas evoluções para fixar o conteúdo.

1

Criar policy PodeExcluir

Somente Admin com claim estoque=excluir.

2

Adicionar filtro por quantidade mínima

GET /produtos?quantidadeMinima=10.

3

Adicionar endpoint DELETE

Proteger com a nova policy.

4

Registrar auditoria no DELETE

Salvar usuário, operação e produto.

5

Testar 401, 403 e 204

Com tokens diferentes no Swagger.

Perguntas de revisão

Use no final para medir entendimento da turma.

1. JWT é criptografado ou assinado?

2. Qual a diferença entre 401 e 403?

3. Role e Claim são a mesma coisa?

4. O que uma Policy faz?

5. CORS protege contra usuário sem permissão?

6. Por que UseAuthentication vem antes de UseAuthorization?

7. Health check e métricas são a mesma coisa?

8. Por que segredo não deve ficar no Git?

Resumo final

Backend moderno não é só endpoint funcionando.



Arquitetura

Camadas claras reduzem acoplamento.



Segurança

JWT, policies, roles, claims e HTTPS.



Prontidão

Erros, auditoria, métricas, health e rate limit.

Prof. Lucas Lima